

Medicare

Architecture and Design Analysis Report

Name: Hitanshu Bhatt

Student ID: 000367185

Cancer Detection Capstone II Project

Prepared from the submitted project repository and UI assets

Project type	Full-stack medical web application with image-classification ML component
Core architecture	FastAPI backend + static frontend + MySQL + PyTorch model
Primary dataset	Lung histopathology image folders with 4 output classes
Report focus	Architecture selection, interface contracts, stack justification and design patterns

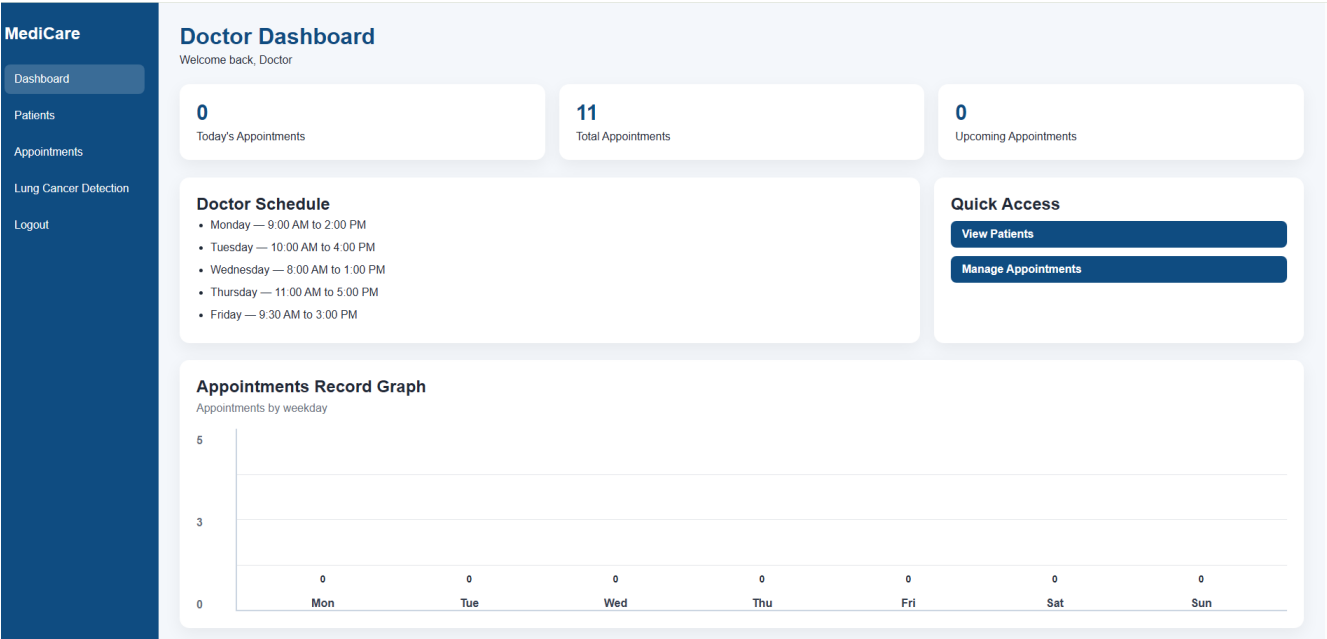


Figure 1. Doctor dashboard interface used as the main operational view for clinicians.

1. Executive Summary

This report evaluates the submitted LungVision Pro project as a deployable capstone system rather than only as a machine-learning notebook. The repository contains a browser-based doctor workflow, a FastAPI backend, MySQL persistence, and an EfficientNet-B0 image classifier for four-way lung-cancer categorization. The selected architecture is best described as a layered monolith: presentation pages call one REST API, the API coordinates both database operations and inference services, and a single relational database stores operational records.

The strongest architectural decision in the project is the choice to keep the first release simple. For a student capstone with one primary developer, a monolithic deployment minimizes operational overhead, keeps debugging manageable, and allows fast integration across doctor registration, appointments, patient data, and model inference. The main trade-off is that tighter coupling between frontend expectations and backend routes has already produced contract gaps that would need to be resolved before production use.

From the code inspection, the solution already demonstrates several good software-engineering habits: modular ML loading through dedicated service files, use of Pydantic request models, normalized image preprocessing, and a clear separation between training scripts and runtime inference. However, the implementation still requires contract completion and security hardening, especially around authentication, credential management, and endpoint consistency.

2. Evidence Collected from the Repository

Repository area	Observed files	Architectural implication
Frontend	HTML, CSS and JS pages for login, dashboard, patients, appointments, booking and detection	Confirms browser-based presentation layer with manual fetch integration
Backend	FastAPI main.py, database helper, prediction router, model loader and prediction service	Confirms API layer and modular ML inference path
Training	train.py, lung_model.py, test.py, saved .pth model	Confirms offline training workflow separated from deployed inference
Data assets	Datasets/train, validation and test folders; screenshots/	Supports ML training and report visuals

Dataset folders indicate a 4-class image-classification problem with the following visible file counts in the supplied repository: training = 613 images, validation = 72 images, and test = 315 images. The training split is somewhat imbalanced, which makes the use of transfer learning on EfficientNet-B0 reasonable for achieving useful performance with limited data.

3. Architecture Selection and Trade-off Analysis

Three realistic architectural approaches fit this project: a layered monolith, a microservices design, and a strictly separated SPA-plus-API system using a heavier front-end framework. The implemented code most closely matches the layered monolith, and for this capstone that is the most defensible choice.

Approach	Fit for this project	Advantages	Disadvantages	Overall verdict
Layered monolith	High	Single deployment, fast debugging, low ops overhead, easier integration for one team	Can become tightly coupled; scaling all modules together; route drift can affect multiple pages	Best choice for capstone MVP
Microservices	Low to medium	Independent scaling of auth, booking and ML inference; stronger team parallelism	Too much complexity for student scope; needs service discovery, monitoring	Over-engineered for current scope

			and container orchestration	
SPA + separate backend	Medium	Richer component reuse and stronger client-side state control	Adds framework learning cost; more build tooling; still needs the same backend contracts	Good future evolution, not necessary for first release

Why the layered monolith was the correct choice:

- Scalability needs are modest at capstone stage. Appointment volume and inference traffic can be handled on one backend process or a small number of replicated instances.
- Team size is effectively one developer, so a unified codebase reduces coordination cost and keeps change impact visible.
- Deployment is simpler. A FastAPI server, a MySQL instance and static pages are easier to demonstrate and maintain than multiple networked services.
- Feature coupling is naturally high because diagnosis, booking and patient context are used together in the same doctor workflow.

Trade-offs identified:

- The backend mixes database access styles: aiomysql is used in signup while mysql.connector is used elsewhere. This weakens architectural consistency.
- Several frontend pages call routes that are not present in the inspected main.py file, including /login, /patients, /register-patient and /add-appointment. This is a contract mismatch risk.
- The ML inference path is modular, but it still ships inside the same application process. Under heavy GPU/CPU load, diagnostic inference could affect operational API latency.

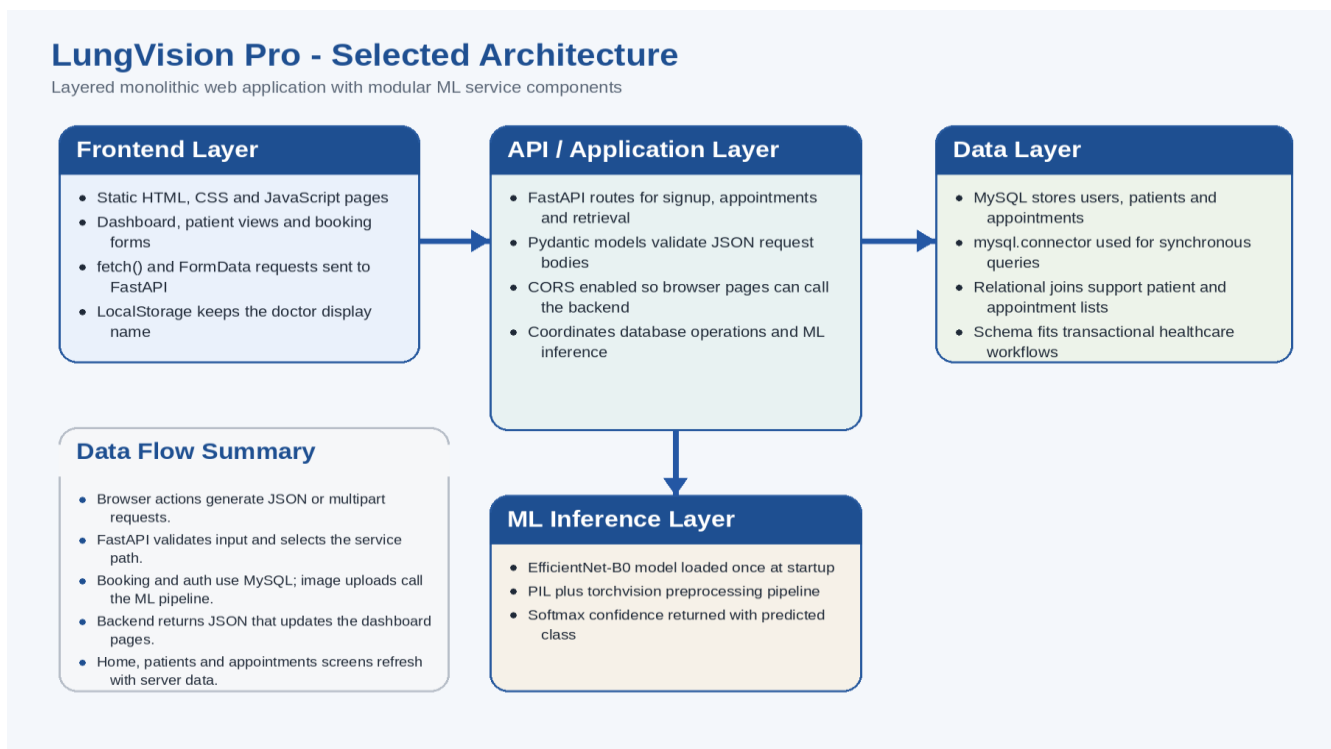


Figure 2. Selected layered-monolith architecture extracted from the code structure and runtime flow.

4. Interface Contracts

The project is integration-heavy, so interface contracts are critical. This section documents the observable API endpoints, request shapes, response expectations, and frontend-to-backend touchpoints based on the submitted code.

Endpoint	Method	Request schema	Response	Status
/	GET	None	{"message": "Lung Cancer Detection API Running"}	Implemented
/signup	POST	full_name, email, password, dob, address	Success message after doctor creation	Implemented
/public-book-appointment	POST	email, appointment_time, reason?	Success or booking conflict error	Implemented
/create-patient	POST	first_name, last_name, dob, email, phone	Success or already exists message	Implemented
/get-appointments	GET	None	Appointment list joined with patient names	Implemented
/predict-lung-cancer	POST multipart	file: image upload	prediction label + confidence	Implemented
/login	POST	email, password	Doctor identity for localStorage	Expected by frontend, not observed
/patients	GET	None	Patient list for dropdowns and tables	Expected by frontend, not observed
/register-patient	POST	Expanded patient profile payload	Registration confirmation	Expected by frontend, not observed
/add-appointment	POST multipart	patient_id, doctor_id, appt_date, appt_time, reason	Save confirmation	Expected by frontend, not observed

Key data schemas in use:

- UserSignup schema: full_name: str, email: EmailStr, password: str, dob: str, address: str.
- PublicBooking schema: email: str, appointment_time: str, reason: Optional[str].
- PatientCreate schema: first_name: str, last_name: str, dob: str, email: str, phone: str.
- Prediction response schema: { prediction: string, confidence: float }.

Component interfaces and integration points:

- detection.js sends FormData to /predict-lung-cancer and expects prediction/confidence JSON to update the DOM.
- home.js requests /get-appointments and derives dashboard statistics in the client layer.
- appointments.js expects patient lookup data from /patients and submits multipart appointment creation to /add-appointment.
- patients.js expects a richer patient registration contract than the currently visible PatientCreate model supports.

Interface Contract Map

Observed integration points between pages, endpoints and backend services

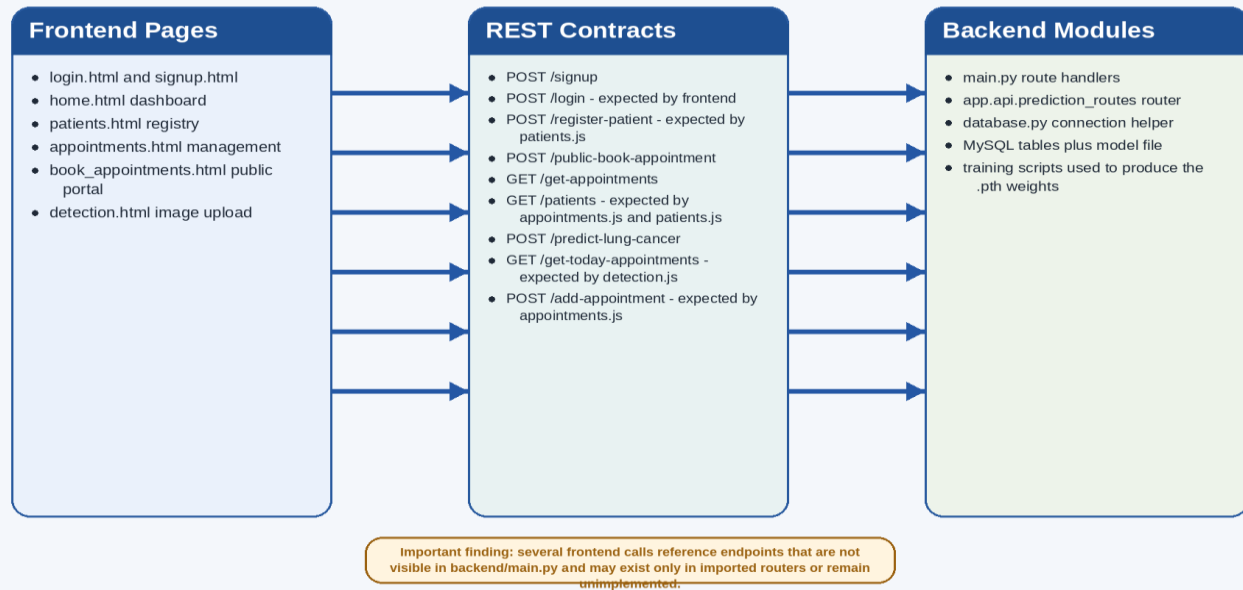


Figure 3. Interface map showing both implemented and expected integration points.

5. Technology Stack Justification

Technology	Why it fits this project	Alternative considered	Assessment
FastAPI	Strong typing via Pydantic, fast endpoint development, built-in docs, good fit for Python ML integration	Flask or Django	FastAPI is the strongest fit because the same language can serve both API logic and ML code
Vanilla HTML/CSS/JS	Low learning curve, fast for capstone prototyping, no bundler needed for static pages	React or Vue	Acceptable for MVP, but harder to maintain as UI complexity grows
MySQL	Relational model suits users, patients and appointments; joins are straightforward and data is transactional	MongoDB or PostgreSQL	Relational choice is correct; PostgreSQL would also be strong, but MySQL is simpler and commonly taught
PyTorch + torchvision	Excellent support for transfer learning and EfficientNet-B0; direct compatibility with training and inference scripts	TensorFlow/Keras	Good choice given model loading and preprocessing already use PyTorch
EfficientNet-B0	Efficient transfer-learning backbone with solid accuracy-to-compute ratio for medium-size image tasks	Custom CNN or larger EfficientNet	Appropriate balance between performance and training cost
localStorage for session name	Very easy for demo flow and page personalization	JWT or secure cookie sessions	Only suitable for demonstration, not for production security

Learning curve and compatibility observations:

- React or Vue would improve component reuse and state handling, but they would also add build tooling and framework conventions that may distract from a capstone deadline.
- MySQL is compatible with the CRUD-heavy healthcare workflow. The main improvement needed is better connection management and secrets handling rather than a database swap.

- Python across training and serving is a major advantage. It avoids serialization friction and reduces the number of languages needed to keep the model in production.

6. Design Patterns Observed or Strongly Applicable

Pattern	Where it appears or should appear	Reason it matters
Layered architecture	Frontend files, FastAPI routes, database access and ML service modules are separated into distinct responsibilities	Improves maintainability and keeps UI, business logic and data access conceptually distinct
Model-Service pattern	model_loader.py and lung_prediction_service.py isolate model initialization and prediction behavior	Prevents repeated model loading and keeps inference logic reusable
MVC-style separation	HTML pages act as views, JS files act as controllers, and MySQL/Pydantic objects serve as model representations	Helps explain how user interactions are transformed into API requests and rendered results
Factory-like object creation	Pydantic models create validated request objects for endpoint handlers	Provides predictable contract validation at service boundaries
Observer/event-driven UI	DOMContentLoaded listeners and form submit handlers react to user events and API responses	Keeps browser screens synchronized with backend data

Pattern relationships:

The layered architecture is the top-level structural pattern. Inside it, MVC-style page organization explains UI behavior, while the model-service pattern isolates the ML subsystem from route handlers. Event-driven browser code acts as the glue between views and REST contracts. Together these patterns make the system understandable even though it is not yet fully formalized into a larger enterprise framework.

7. Supporting UI Evidence

The submitted screenshots confirm that the solution is designed as an operational product, not only as a backend or notebook demonstration. The UI covers dashboard analytics, appointment management, and a dedicated diagnostic page for image upload.

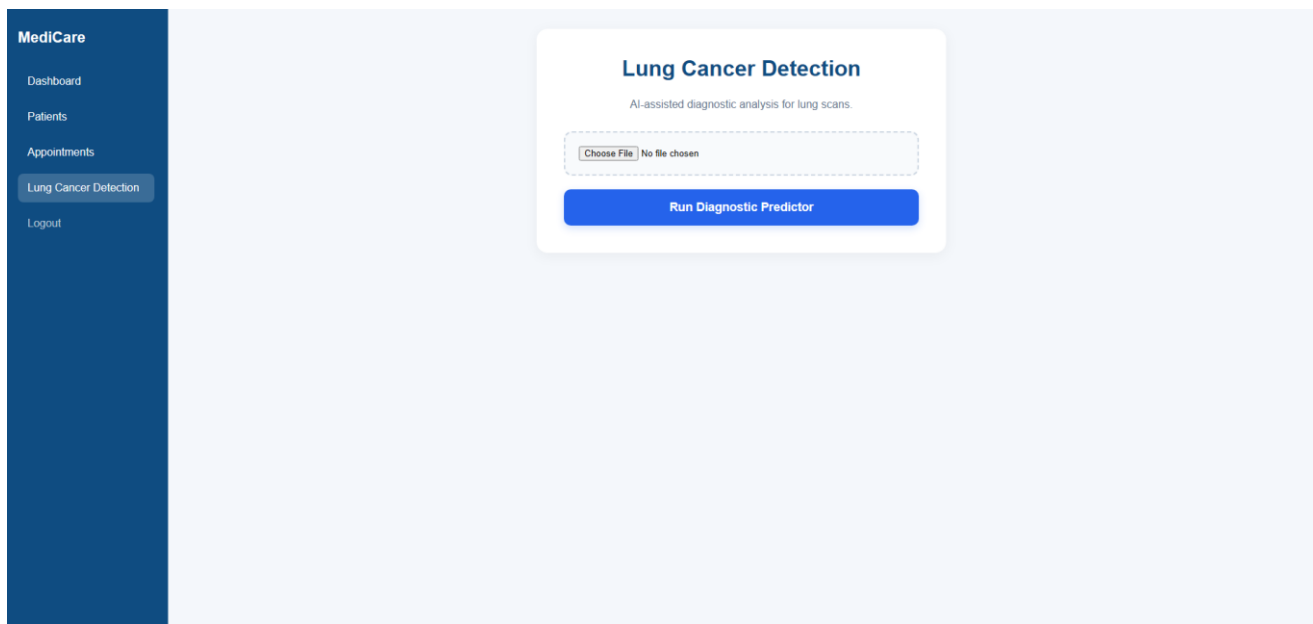


Figure 4. Diagnostic interface for uploading a scan and showing predicted class/confidence.

Figure 5. Appointment-management screen demonstrating workflow support beyond pure AI prediction.

Conclusion

Overall, the submitted project makes a sound architectural choice for a capstone: a layered monolithic application that keeps the machine-learning path integrated with a practical doctor's workflow. That choice matches the project size, available team capacity, and demonstration goals. The stack is coherent, and the modular inference code is one of the strongest design decisions in the repository.

The main issue is not the high-level design but contract completion. Several frontend modules assume routes and data fields that are not visible in the inspected backend. That means the architecture is conceptually correct, but the implementation contracts still need one final alignment pass before the system can be considered fully complete.

Priority improvements before final submission or deployment:

- Unify the API contract so every frontend fetch target has a matching backend route and schema.
- Move database credentials out of source code into environment variables and use stronger password handling than plain SHA-256.
- Standardize database access to one library and add service/repository helpers for cleaner data access.
- Add explicit test cases for login, patient registration, booking collision checks and prediction endpoint error handling.
- Consider evolving the ML module into a separately scalable service only after the monolithic version is stable and validated.

In conclusion, the selected architecture is justified and appropriate. With endpoint alignment, security hardening, and modest refactoring, this capstone can be presented as a credible end-to-end healthcare software system that combines web engineering with applied deep learning.